LR-Agent / Agent 后端入门

第02章
LangChain模型消息与链入门

先理解一次模型调用的输入与输出，再把 Prompt、模型和解析器接起来。最后回到项目，区分固定链与 Agent。

读者起点

只掌握最基本的 Python；LangChain 从零开始。每次先读一个小例子，运行并解释结果，再进入项目源码。

学完本章应该能够

- 说清模型、API、框架、消息与 Prompt 的基本关系。
- 运行消息、固定模型、模板、LCEL 与 stream 的完整实验。
- 读懂项目请求怎样转换成 LangChain 消息。
- 解释固定链、Agent 和 RAG 的职责区别，形成有边界的面试表述。

范围：Python、LangChain 与 Agent 后端。外部调用方只作为接口参与者，不安排前端实现学习。
基线：2026-10-17，d5dd009；LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与阅读方法

2.1 从普通函数走向模型调用 . . . 3

2.2 Message：先把对话变成对象 . . . 4

2.3 ChatModel 与 invoke：一次输入，一次结果 . . . 5

2.4 项目怎样构造真实模型 . . . 6

2.5 Prompt 模板：变化的内容放在哪里 . . . 7

2.6 Runnable：统一的执行约定 . . . 8

2.7 完整实验：用 LCEL 接起固定链 . . . 9

2.8 固定链和 Agent，差别在哪里 . . . 10

2.9 stream：从一次结果变成多个片段 . . . 11

2.10 ToolMessage：先认识调用与结果的关系 . . . 12

2.11 从真实请求 Schema 进入项目 . . . 13

2.12 顺着源码找到官方 Agent 工厂 . . . 14

2.13 用输入输出定位常见失败 . . . 15

2.14 练习、面试追问与进入下一章 . . . 16

每节按“具体问题 → 名词解释 → 小例子 → 输出解释 → 项目对应 → 练习”推进。若代码出现陌生语法，先返回第 1 章相应小节，不要求提前掌握整门 Python。

蓝色标题可通过目录或书签跳转。完整实验由读者自行新建文件，不依赖额外附件。源码节选用于阅读，不能直接当完整脚本运行。

2.1 从普通函数走向模型调用

第 1 章的 count_images 根据列表长度确定地计算数量。如果用户写“把这组照片里的汽车框出来”，需要先把自然语言变成可执行意图；模型可以参与理解和提出调用，但不会仅凭一句话直接完成本机文件操作。

名词	一句话解释	本项目里的角色
LLM / 大语言模型	根据上下文生成内容的模型	理解任务、回答或提出工具调用
API / 接口	调用服务时约定的输入输出	后端向模型服务发送请求
LangChain	组织模型、消息、工具等调用的框架	统一对象和执行接口
LangGraph	组织图执行与状态流转的框架	create_agent 的底层执行基础
Prompt / 提示	本次给模型的任务说明与上下文	约定角色、规则和用户问题

LangChain 不等于模型本身

模型服务提供推断能力，LangChain 帮助程序组织调用。替换框架并不会凭空换掉模型知识；安装 LangChain 也不会自动获得可用的云模型账号或执行权限。

Token：模型处理文本或其他内容时使用的计量单位，不严格等于一个汉字或一个单词。上下文窗口和输出预算通常按 token 计量；本章不假设具体模型的价格或上限。

本章先用固定模型隔离网络与回答不确定性。真实模型的兼容、延迟和错误处理，在项目适配与调试章节再验证。

2.2 Message：先把对话变成对象

模型通常接收一组消息。消息不只有文本，还包含角色及可能的工具调用等信息。HumanMessage 表示用户输入，AIMessage 表示模型响应，SystemMessage 提供规则和身份说明。

完整教学实验 lc_messages。先在 study 中自行新建 my_lc_messages.py，复制全部代码。

```python
from langchain_core.messages import (
    SystemMessage, HumanMessage, AIMessage,
)

messages = [
    SystemMessage(content="Explain annotation tasks briefly."),
    HumanMessage(content="How many images?"),
    AIMessage(content="There are 2 images."),
]
for message in messages:
    print(message.type, "->", message.content)
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_lc_messages.py"
```

本轮实际输出：

```
system -> Explain annotation tasks briefly.
human -> How many images?
ai -> There are 2 images.
```

messages 是列表，每一项是对象；message.type 读取角色类型，message.content 读取内容。API 中常说 user / assistant，LangChain 对象的 type 通常显示 human / ai，不能只按字面名称判断是否同一角色。

例子没有发起模型请求

AIMessage 是在代码里手工构造的历史回答。创建一个消息对象不等于向模型调用一次；真正的调用将在下一节出现。

练习：增加第二条 HumanMessage，观察遍历顺序。请解释为什么仅把所有文本拼成一个字符串，会丢失一部分角色结构。

2.3 ChatModel 与 invoke：一次输入，一次结果

ChatModel 表示聊天模型对象。invoke 是 LangChain 常见的一次执行入口。输入消息，返回 AIMessage；调用者通常继续读取 content，或检查是否携带工具调用。

完整教学实验 lc_fake。先在 study 中自行新建 my_lc_fake.py，复制全部代码。

```python
from langchain_core.language_models.fake_chat_models import (
    FakeListChatModel,
)
from langchain_core.messages import HumanMessage

model = FakeListChatModel(responses=["There are 2 images."])
response = model.invoke([HumanMessage(content="Count a.jpg and b.jpg.")])
print("response type:", response.type)
print("content:", response.content)
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_lc_fake.py"
```

本轮实际输出：

```
response type: ai
content: There are 2 images.
```

responses 中的答案由我们事先写好。model.invoke 确实走了 LangChain 模型接口，但回答内容不是模型理解问题后推断出来的。response 是对象，不是普通字符串，所以读取 .content。

为什么先用固定模型

你可以专心观察参数、返回类型和执行顺序。将输入改成三张图片，固定回答仍是两张，恰好说明它只能测试框架和代码，不能评价真实模型的推理能力。

项目对应：entry.py 构造的真实模型也遵循 ChatModel 相关接口；业务工厂通过参数接收模型，便于测试时替换为固定模型。

2.4 项目怎样构造真实模型

源码位置：app/agent/langchain_runtime/entry.py::build_llm_pair。它接收已校验的请求 body，使用 ChatOpenAI 兼容客户端构造主模型，条件满足时再构造辅助模型。

源码节选。factory 已在前文绑定为模型构造器，body 和 base_url 由调用上下文提供；此段不作为完整脚本运行。

```
llm = factory(
    model=body.model,
    api_key=body.api_key,
    base_url=base_url,
    streaming=True,
    temperature=0.7,
    timeout=120,
    extra_body=_deepseek_extra_body(body.model, base_url),
)
```

参数	含义	阅读时注意
model / base_url	模型标识与服务地址	兼容客户端不代表所有服务能力一致
api_key	请求模型服务所需凭据	不要写进教材、日志或 Git
streaming	启用客户端流式调用配置	不等于已经向用户发送 SSE
temperature	采样相关配置	不能简单理解为准确率开关
timeout	调用超时配置	不是整个业务任务的预算

构造实例与执行请求是两个阶段。build_llm_pair 主要装配对象；后续 invoke/astream 才进入执行。项目还带有服务适配分支，不能把某个模型的兼容策略照搬给所有提供方。

2.5 Prompt 模板：变化的内容放在哪里

如果每次都手工拼接整段提示，字段容易遗漏。ChatPromptTemplate 保存消息结构，用 {count} 和 {label} 标出变量；执行时传入字典，得到填好内容的消息值。

完整教学实验 lc_prompt。先在 study 中自行新建 my_lc_prompt.py，复制全部代码。

```python
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "Explain the annotation request briefly."),
    ("human", "Images: {count}; label: {label}"),
])
value = prompt.invoke({"count": 2, "label": "car"})
print("variables:", sorted(prompt.input_variables))
for message in value.to_messages():
    print(message.type, "->", message.content)
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_lc_prompt.py"
```

本轮实际输出：

```
variables: ['count', 'label']
system -> Explain the annotation request briefly.
human -> Images: 2; label: car
```

("human", "文本") 这种小括号包住的两项叫元组，此处按“角色、文本模板”解释。prompt.invoke 返回 PromptValue；.to_messages() 转成消息列表。这里只完成了模板替换，没有调用模型。

不要用“优化提示词”概括所有问题

变量名拼错是输入组织问题；权限或重复写入是业务约束问题；模型服务超时是调用问题。提示写得更长，不能替代代码校验和执行控制。

练习：只传 count 不传 label，观察缺少变量时报错。先定位缺失字段，再决定是否需要默认值，而不是把异常当成模型回答失败。

2.6 Runnable：统一的执行约定

Runnable 可以先理解为“遵守统一执行接口的组件”。模板、模型、解析器都能参与组合。它们接受和返回的数据类型不同，但可以通过约定连接起来。

组件	输入	输出
ChatPromptTemplate	包含 count、label 的字典	PromptValue / 消息内容
ChatModel	消息或可转换的提示内容	AIMessage
StrOutputParser	模型消息或文本	普通字符串



图 2-1 固定链的数据类型流转示意。

invoke 通常返回一次结果；ainvoke 是异步等待一次结果；stream/astream 用于逐步产出。它们描述调用方式，不直接决定业务是否使用 Agent 或是否存在多个模型。

2.7 完整实验：用 LCEL 接起固定链

LCEL 是 LangChain Expression Language 的缩写。这里使用 | 把模板、模型和解析器连接起来。这是框架为对象定义的组合行为，不是系统终端里的管道命令。

完整教学实验 lc_chain。先在 study 中自行新建 my_lc_chain.py，复制全部代码。

```python
from langchain_core.language_models.fake_chat_models import (
    FakeListChatModel,
)
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("human", "Images: {count}; label: {label}"),
])
model = FakeListChatModel(responses=["2 images; label car."])
parser = StrOutputParser()
chain = prompt | model | parser
result = chain.invoke({"count": 2, "label": "car"})
print("result:", result)
print("is string:", isinstance(result, str))
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_lc_chain.py"
```

本轮实际输出：

```
result: 2 images; label car.
is string: True
```

chain.invoke 先接收字典，模板填充后交给模型，模型返回消息，再由解析器返回 str。StrOutputParser 只处理输出形式，没有验证“两张图片”是否真实。

isinstance(result, str) 检查结果能否作为字符串使用，也接受字符串子类。当前环境中可能看到 TextAccessor 类型，它继承 str；现在只需理解输出可按文本处理，不必追入库内部。

这是完整离线教学链，LR-Agent 当前默认 Agent 内核并不是直接用这条 LCEL 链实现。学习它是为了掌握框架的数据组合，再理解 Agent 的额外控制逻辑。

2.8 固定链和 Agent，差别在哪里

假设任务总是“整理用户给的标注需求”，固定的模板 → 模型 → 文本解析通常容易理解和控制。若任务需要先查看目录，再决定读哪些文件或调用哪种工具，就需要额外的决策与执行循环。

比较点	本章固定链	项目 Agent 路径
执行顺序	写代码时就确定	模型响应可能要求调用工具后继续
执行内容	模板、一次模型与解析	模型、工具、消息更新和业务约束
控制责任	调用者组织固定流程	官方循环与业务中间件共同约束
失败排查	检查各组件输入输出	还要检查调用 ID、预算、状态与副作用

为什么不是所有任务都用 Agent

当步骤固定时，增加模型决策会带来额外的不确定性与排查成本。任务确实需要选择工具或根据结果继续判断，才更值得学习 Agent 的循环与约束。

ReAct 先只理解思想

Reasoning + Acting 强调推理与行动交替。当前工具调用的实际控制要看框架消息和业务代码，不能因为名字含 ReAct 就声称能读取模型隐藏思考或完整复现某种提示流程。

面试练习：先举一个适合固定链的任务，再举一个需要根据工具结果继续选择的任务，分别说明取舍。

2.9 stream：从一次结果变成多个片段

模型流式接口逐步产出消息片段。业务代码可以处理每个片段并拼接最终内容。片段边界由具体实现决定，不能普遍假设一片恰好等于一个字或一个 token。

完整教学实验 lc_stream。先在 study 中自行新建 my_lc_stream.py，复制全部代码。

```python
from langchain_core.language_models.fake_chat_models import (
    FakeListChatModel,
)

model = FakeListChatModel(responses=["OK"])
print("invoke:", model.invoke("ready?").content)
chunks = []
for chunk in model.stream("ready?"):
    print("chunk:", repr(chunk.content))
    chunks.append(chunk.content)
print("joined:", "".join(chunks))
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_lc_stream.py"
```

本轮实际输出：

```
invoke: OK
chunk: 'O'
chunk: 'K'
joined: OK
```

本例 FakeListChatModel 按字符产生 O、K，这是它的教学实现。真实服务的切分可能不同。join 负责将字符串片段连接；它没有增加新的模型调用。

模型流与 SSE 是两层

stream 表示模型/框架如何产出数据；SSE 表示 HTTP 响应如何把事件送给调用方。项目还要做事件适配、错误和结束处理，后续第 5 章展开。

练习：把固定回答改成 READY，预测 invoke 与 stream 分别打印什么。异步版本 astream 还需要 async for，先不要混写普通 for。

2.10 ToolMessage：先认识调用与结果的关系

模型可能返回工具调用，而不只是一段回答。AIMessage.tool_calls 描述名称、参数和 ID；ToolMessage 携带结果，并通过 tool_call_id 指回原调用。

完整教学实验 lc_tool_message。先在 study 中自行新建 my_lc_tool_message.py，复制全部代码。

```python
from langchain_core.messages import AIMessage, ToolMessage

planned = AIMessage(content="", tool_calls=[{
    "name": "count_images", "args": {"paths": ["a.jpg", "b.jpg"]},
    "id": "call-001", "type": "tool_call",
}])
result = ToolMessage(content="2", tool_call_id="call-001")
call = planned.tool_calls[0]
print("tool:", call["name"])
print("matched:", call["id"] == result.tool_call_id)
print("result:", result.content)
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_lc_tool_message.py"
```

本轮实际输出：

```
tool: count_images
matched: True
result: 2
```

content 为空但 tool_calls 非空，是合法的工具调用消息。call-001 用来关联结果；工具名只是能力标识，不能代替调用 ID，因为同一工具可能调用多次。

这里没有执行工具

代码只构造一组相互匹配的消息，结果 2 是手写的。第 3 章将使用真正的工具函数和 create_agent，观察执行节点怎样生成 ToolMessage。

练习：把结果 ID 改为 call-002，matched 将变为 False。这说明消息不匹配；真实接口怎样拒绝或整理输入，要看项目协议与续跑实现。

2.11 从真实请求 Schema 进入项目

现在把前面学到的 Pydantic 与消息对象连起来。项目 LocalChatStreamRequest 描述一次请求；_build_lc_messages 把请求中的消息转换成 LangChain 对象。

完整教学实验 lc_request。先在 study 中自行新建 my_lc_request.py，复制全部代码。

```python
from app.schemas.agent import LocalChatStreamRequest, ChatMessageInput
from app.agent.langchain_runtime.entry import _build_lc_messages

body = LocalChatStreamRequest(
    api_key="offline-placeholder",
    base_url="http://localhost/unused",
    model="offline-model",
    messages=[ChatMessageInput(role="user", content="Count 2 images.")],
    user_content="Count 2 images.",
    client_job_id="learning-001",
)
messages = _build_lc_messages(body, "Explain tasks briefly.")
for message in messages:
    print(message.type, "->", message.content)
print("job:", body.client_job_id)
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_lc_request.py"
```

本轮实际输出：

```
system -> Explain tasks briefly.
human -> Count 2 images.
job: learning-001
```

offline-placeholder 只是满足非空字段的假值，localhost/unused 也不会被访问。此例只创建请求对象并调用纯消息转换；client_job_id 用来标识业务任务，不等于 LangGraph 的持久化线程 ID。

_build_lc_messages 是内部函数，本实验用于学习当前实现，不把它当作稳定公共 API。本例保留了完整必填字段；user_content 与 messages 在请求层是两个独立字段。

2.12 顺着源码找到官方 Agent 工厂

不用立即读完入口文件。先找到三个符号，分别问：模型怎样创建？消息怎样转换？工具与约束怎样交给官方工厂？

文件（相对 local-agent）	符号	这一层负责什么
app/schemas/agent.py	LocalChatStreamRequest	请求字段与结构校验
app/agent/langchain_runtime/entry.py	build_llm_pair _build_lc_messages	模型实例与消息转换
app/agent/langchain_runtime/entry.py	build_engine_agent	工具、中间件、上下文装配
app/agent/langchain_runtime/runtime.py	build_langchain_agent	调用官方 create_agent

源码节选：runtime.py::build_langchain_agent 的返回部分；变量来自其函数参数。

```
return create_agent(
    model,
    list(tools),
    system_prompt=system_prompt,
    context_schema=context_schema,
    name=name,
    middleware=list(middleware),
)
```

这段不是 Prompt | 模型 | 解析器。它构建包含模型与工具控制逻辑的图，调用方再去执行。当前代码没有传 checkpointer 或 Store，不能把课程中的持久化恢复视为已经启用。

当前默认配置名是 candidate；legacy 是回退路径。普通聊天在不具备工作区工具条件时可能走直接聊天流，不能说每次请求都会执行完整 Agent。

2.13 用输入输出定位常见失败

现象	先检查什么	不能直接得出的结论
模板提示缺少变量	invoke 字典键是否齐全	不能直接归因模型不支持
输出还是固定答案	是否使用了 FakeListChatModel	不能当成模型幻觉测试
拿到 AIMessage 不是字符串	读取 content 或使用解析器	不能认为调用失败
模型输出工具调用	是否有工具执行与结果回传	不能认为操作已经完成
流接收一半就结束	结束、错误、取消与响应生命周期	不能仅凭部分文本判成功

RAG 与本项目的边界

RAG / Retrieval-Augmented Generation，检索增强生成，通常先检索外部知识，再将相关内容用于生成回答。它主要解决知识获取问题；本项目侧重工作区工具执行和标注确认。

Agent 可以在需要时调用检索工具，但 Agent 并不必然需要向量数据库。本课程不向本项目增加 RAG。课程中的“拆问题、看结果、做对照”的讲法，可以用于解释后端协议和工具效果。

评价也要分层

代码和协议正确、模型理解正确、检测框准确、用户更省时间，是不同问题。分别收集证据，不把本章固定模型的成功输出写成精度或效率提升。

2.14 练习、面试追问与进入下一章

练习 1：把 LCEL 的输入改为 3 张图，固定回答仍为 2，说明什么？
参考：固定模型不根据语义计算；它帮助测试数据流，不能证明模型能力。

练习 2：去掉 StrOutputParser，结果对象与原来有什么不同？
参考：模型通常返回 AIMessage；原链通过解析器返回文本。请亲自修改并打印类型。

练习 3：Prompt.invoke、Model.invoke、Agent.invoke 都叫 invoke，它们做一样的事吗？
参考：统一的是执行入口；输入输出及内部流程由具体组件决定。

30 秒面试解释：为什么用 LangChain

“它提供统一的模型、消息和工具接口，便于装配与测试。项目用官方 create_agent 承接模型和工具循环，再加入业务约束。我先用固定模型验证消息和执行顺序，再与真实服务调试分开。”完成后续 Agent 学习后，再补充具体取舍与本人修改。

1 分钟项目追问：你具体完成了什么

现阶段可据实说明自己运行了哪些入门实验、定位了哪些源码，尚不能把项目全部能力写成独立开发。后续应选择一个具体后端问题，留下修改、测试与失败边界，才形成个人贡献。

通过标准

能够画出字典 → Prompt → 消息 → 模型 → AIMessage → 字符串的数据流，并指出在哪一步没有发生工具执行。之后进入第 3 章学习真正的工具和 Agent 循环。

本章 7 个完整离线实验已在正式环境运行。源码依据：app/schemas/agent.py、langchain_runtime/entry.py 与 runtime.py；依赖以当前 requirements 和虚拟环境为准。